

1. Introduction

Web applications form the backbone of modern digital services, handling everything from financial transactions to personal communication and healthcare records. As their usage has grown, so has their exposure to attackers who exploit weaknesses in how these applications accept and process user input. Among the most persistent and damaging of these weaknesses are SQL Injection (SQLi) and Cross-Site Scripting (XSS) attacks, both of which have consistently featured in the OWASP Top 10 list of web application security risks for over a decade.

Traditional defenses rely heavily on signature-based or regular-expression (regex) filtering, where known malicious patterns are explicitly listed and blocked. Such approaches are inherently brittle: they can only catch attack patterns that have already been anticipated, and are frequently bypassed through encoding tricks, case variation, or novel payload structures.

This project proposes an Adaptive Security Gateway: a full request-inspection pipeline whose detection core is a deep learning model rather than static rules or classical feature-engineered classifiers. The detection engine is built in PyTorch and trained on a CUDA-enabled GPU, using a character-level neural network (CNN/BiLSTM) that learns directly from raw payload text rather than relying on hand-picked keyword or n-gram features. Beyond classification alone, the system is architected as a complete gateway — covering request parsing, threat-intelligence enrichment, session and device tracking, behavioral anomaly scoring, risk-based decisioning, policy enforcement, incident logging, and an analyst-facing dashboard. This mirrors the structure of production-grade security gateways, with the key improvement that the detection core learns underlying patterns in attack payloads rather than matching only what a rule author anticipated in advance.

2. Problem Definition

Existing signature/regex-based gateways fail to reliably detect obfuscated or previously unseen SQL Injection and Cross-Site Scripting payloads. This project addresses that gap with a deep learning-based detection core embedded in a complete security gateway pipeline.

3. Objectives & Scope

Objectives

- To design and train a deep learning model (character-level CNN/BiLSTM) using PyTorch on a CUDA-enabled GPU, capable of classifying request input as benign, SQL Injection, or Cross-Site Scripting, with a confidence score.
- To build a complete gateway pipeline around the detection core: request parsing, preprocessing, threat intelligence, session management, device profiling, and behavioral anomaly scoring.
- To design a risk-scoring and decision framework that converts model output and contextual signals into an actionable verdict (ALLOW / MONITOR / RATE_LIMIT / BLOCK), with a policy layer for hard overrides on critical attack types.
- To implement incident logging and an analyst-facing dashboard for reviewing flagged requests and confirming or overriding verdicts.
- To evaluate the system's detection reliability using a held-out test dataset and previously unseen example payloads.

Scope

- The gateway can operate in reverse-proxy mode (in front of a protected application) or middleware mode (the application queries the gateway's decision API per request).
- Detection is limited to SQL Injection and Cross-Site Scripting attack categories; other attack types (e.g., command injection, CSRF, SSRF) are noted as future extensions of the same pipeline.
- The ML detection core uses a PyTorch deep learning model trained on a CUDA-enabled GPU; no classical/feature-engineered baseline model is used in the final system.
- Caching is split by purpose: a server-side in-memory cache (Flask-Caching) secures session/rate-limit tracking, while a lightweight client-side cache (node-cache, run locally alongside the React dashboard) handles UI convenience only.
- The system is designed and demonstrated for localhost/single-instance operation at a scale of roughly up to 1,000 users; it is not intended for deployment as a production,

high-availability Web Application Firewall, and does not include containerized deployment or a continuous model-retraining pipeline in the current scope.

4. Literature Review

4.1 Detection of Structured Query Language Injection Attacks Using Machine Learning Techniques (Angula & Hashiyana)

This study compares classical machine learning classifiers — Logistic Regression, Naive Bayes, Decision Tree, Random Forest, KNN, and SVM — for SQL Injection detection, finding Naive Bayes achieved the highest accuracy among the classical models tested. While this project moves beyond classical, feature-engineered classifiers to a deep learning model, this study establishes the accuracy baseline that the PyTorch-based detection engine is measured against, and confirms that even simple lexical patterns carry strong signal for SQLi detection.

4.2 SQL Injection Attack Detection using Machine Learning Algorithms (Aburashed, AL Amoush & Alrefai, 2024)

This paper evaluates Random Forest, Gradient Boosting, SVM, and Artificial Neural Network (ANN) classifiers for SQLi detection, finding that the ANN model achieved the best overall performance and required the least detection time. This result directly supports this project's decision to use a neural network (rather than a classical model) as the detection core, since it demonstrates that a properly trained neural architecture can outperform classical approaches on this exact task.

4.3 A Hybrid Approach for Detecting SQL Injection Using Machine Learning Techniques (Krishna, Oluoch & Kim, 2025)

This work proposes a hybrid pipeline combining Naive Bayes, LSTM-based feature extraction, and a Random Forest classifier to improve accuracy beyond any single algorithm. The use of an LSTM component for feature extraction in this hybrid design directly supports this project's choice of a BiLSTM/CNN-based architecture as the primary detector, since it confirms that sequence-aware neural models capture patterns in SQLi payloads that simpler feature-based methods miss.

4.4 Securing Web Applications Against XSS and SQLi Attacks Using a Novel Deep Learning Approach (Scientific Reports, 2024)

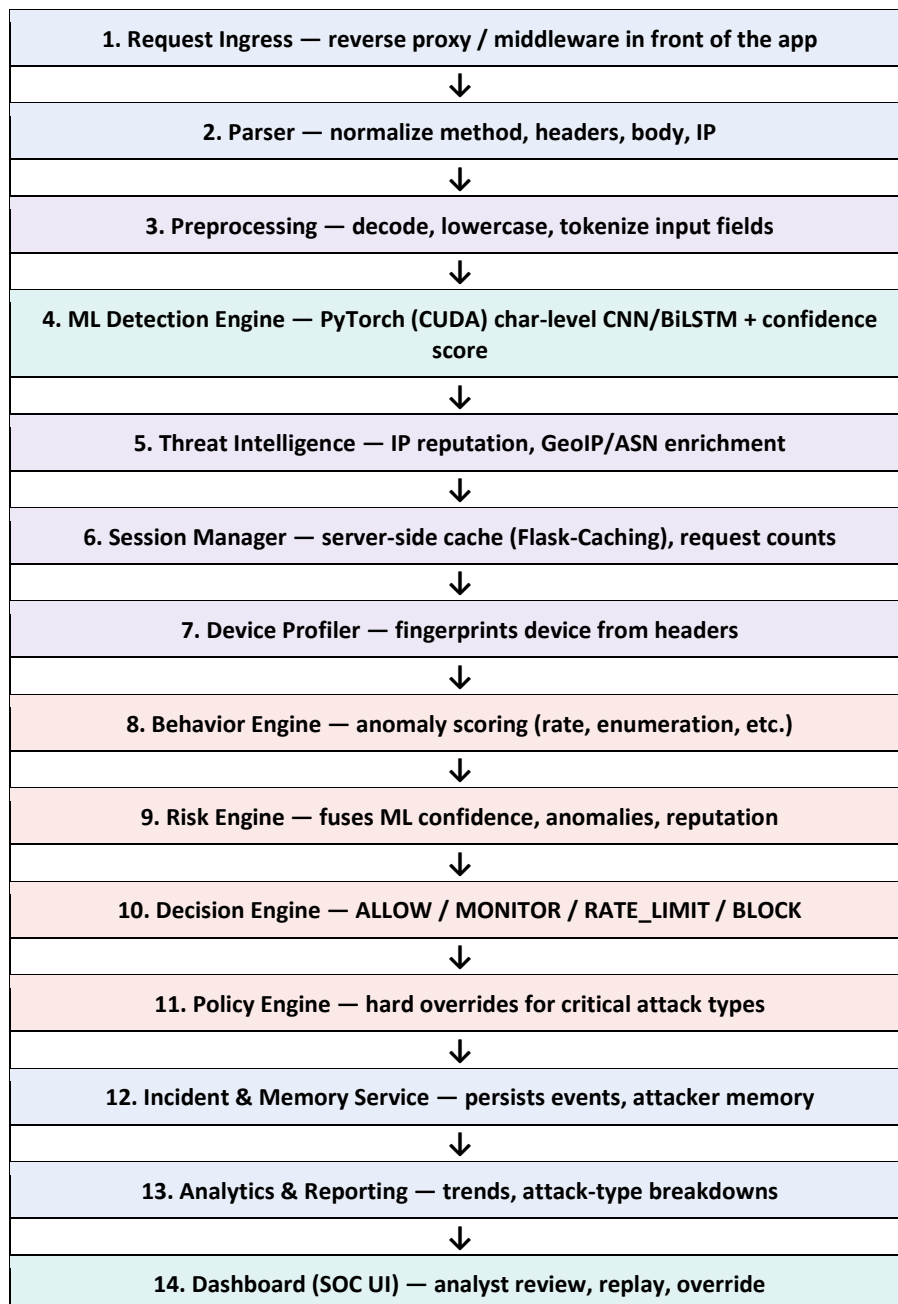
This paper uses the HTTP CSIC 2010 dataset, containing normal and attack-based HTTP requests covering SQLi, XSS, and other categories against a simulated e-commerce application, and applies a deep learning approach to detection. It is the most directly relevant prior work for this project: it confirms that deep learning is an established and effective approach for exactly this SQLi/XSS detection problem, and that full-request-level detection (not just isolated payloads) is a valid design choice, supporting this project's request-ingress pipeline (Stages 1–3) feeding into a deep learning detection core (Stage 4).

4.5 UniEmbed: A Unified Approach for Detecting XSS and SQL Injection Attacks Using Machine Learning Classifiers

This study combines multiple text-embedding techniques (Word2Vec, Universal Sentence Encoder, FastText) as classifier input features for both XSS and SQLi detection, reporting very high accuracy using a Multi-Layer Perceptron — itself a neural network. It supports this project's single, unified deep learning Detection Engine handling both attack types together in one model, and highlights that learned feature representations (as opposed to hand-engineered ones) are critical to detection performance, reinforcing the choice of a character-level neural model that learns its own representations directly from payload text.

5. Methodology

5.1 Complete Gateway Pipeline (14 Stages)



The detection core (Stage 4) is a PyTorch model trained on a CUDA-enabled GPU. Stage 6 (Session Manager) uses a server-side in-memory cache (Flask-Caching) that cannot be influenced by the client, while the Stage 14 dashboard uses a separate, client-side cache (node-cache) purely for UI responsiveness — the two are kept distinct because only one of them is security-relevant.

5.2 Approach and Steps

- Data Collection: Gather labeled datasets of benign inputs, SQL Injection payloads, and XSS payloads (Kaggle, OWASP payload references).
- Data Cleaning & Labeling: Merge datasets into a unified 'text'/label' schema across three classes — benign, sql, xss.
- Preprocessing: Decode URL/HTML-encoded characters, normalize case, tokenize input text at the character level (Stage 3).
- Model Design: Build a character-level CNN/BiLSTM in PyTorch that learns directly from tokenized payload text, without hand-engineered lexical features.
- Model Training: Train the model on a CUDA-enabled GPU; validate `torch.cuda.is_available()` to confirm GPU utilization during training.
- Model Evaluation: Evaluate on a held-out test set using accuracy, precision, recall, F1-score, and a confusion matrix.
- Context Enrichment: Implement Threat Intelligence, Session Manager, and Device Profiler modules (Stages 5–7) to enrich each request with contextual signals.
- Risk & Decision Logic: Implement the Behavior, Risk, Decision, and Policy Engines (Stages 8–11) to convert model output and context into a final verdict.
- System Integration: Wrap the full pipeline in a Flask gateway service; implement Incident & Memory persistence (Stage 12) and Analytics (Stage 13) against MongoDB/PostgreSQL.
- Dashboard: Build a React-based SOC dashboard (Stage 14), run via a local Node process, using node-cache for client-side UI caching.
- Testing: Validate with dataset-derived test samples and manually crafted, previously unseen payloads to assess generalization.

5.3 Use Case Diagram (Description)

Two primary actors interact with the system. The End User (or an upstream application, in middleware mode) submits requests that trigger the use case 'Process Request,' which internally invokes 'Detect Payload,' 'Enrich Context,' 'Score Risk,' and 'Decide Verdict' as included sub-processes. The Security Analyst is a second actor who interacts with the 'Review Incidents' use case on the dashboard, which includes 'Replay Request' and 'Confirm/Override Verdict.' A third, offline use case, 'Train Model,' is performed by the

Developer and includes 'Load Dataset,' 'Preprocess Text,' 'Train Neural Network on GPU,' and 'Evaluate Model.' The trained model artifact forms the interface connecting the offline training workflow to live gateway operation and dashboard review.

6. Resources / Requirements

Hardware Requirements

- Standard laptop/PC — Minimum: Intel i5 or equivalent, 8GB RAM
- Required for model training/inference: a CUDA-enabled NVIDIA GPU (e.g., GTX/RTX series), since the detection core is a PyTorch deep learning model rather than a classical model
- Recommended: Intel i7 or equivalent, 16GB RAM, for running the gateway and dashboard services concurrently alongside GPU-based training

Software Requirements

- Python 3.10 or later
- ML: PyTorch (CUDA-enabled build), pandas, numpy
- Backend/gateway: Flask (Python), REST API
- Server-side session cache: Flask-Caching (SimpleCache, in-memory)
- Frontend: React (SOC dashboard), run via a local Node process
- Client-side UI cache: node-cache (local, alongside the React dashboard)
- Primary data store: MongoDB or PostgreSQL (incidents, sessions, devices, memory, model versions)
- Development tools: VS Code / Jupyter Notebook, Git & GitHub for version control
- Dataset sources: Kaggle (SQL Injection Dataset, XSS Attacks Dataset)
- Testing: Postman (API testing), pytest (unit testing)

7. Expected Outcome

- A trained PyTorch deep learning model (character-level CNN/BiLSTM) classifying input as benign, SQL Injection, or XSS, targeting $\geq 90\%$ accuracy on held-out test data.
- A fully integrated gateway pipeline covering request parsing, context enrichment, risk scoring, decisioning, and policy enforcement — not just isolated payload classification.
- A working incident logging system with an analyst-facing SOC dashboard for reviewing and overriding verdicts.
- A clear evaluation report including accuracy, precision, recall, F1-score, and confusion matrix analysis for the detection core, with confirmation of GPU-accelerated training.
- A complete project report and live demonstration of the end-to-end gateway, suitable for academic evaluation.

8. Applications

- Deployable as a real Web Application Firewall (WAF) layer, either as a reverse proxy or as middleware integrated into an existing application.
- Useful for small-to-medium web applications and startups that need adaptive request protection without dedicated in-house security teams.
- Serves as an educational reference architecture for how deep learning-based detection integrates into a full security operations pipeline (risk scoring, incident response, analyst review).
- The character-level neural detection approach generalizes to other text-based attack categories (command injection, NoSQL injection, CSRF) within the same pipeline.
- Provides a foundation for future extensions such as periodic model retraining or containerized deployment, should the system need to scale beyond its current single-instance design.

9. References / Bibliography

1. Angula, T. J., & Hashiyana, V. Detection of Structured Query Language Injection Attacks Using Machine Learning Techniques. *International Journal of Computer Science and Information Technology*.
2. Aburashed, L., AL Amoush, M., & Alrefai, W. (2024). SQL Injection Attack Detection using Machine Learning Algorithms. *Semarak International Journal of Machine Learning*, 2(1), 1–12. <https://doi.org/10.37934/sijml.2.1.112>
3. Krishna, H., Oluoch, J., & Kim, J. (2025). A Hybrid Approach for Detecting SQL-Injection Using Machine Learning Techniques. *SciTePress*.
4. Securing web applications against XSS and SQLi attacks using a novel deep learning approach. *Scientific Reports* (2024). Nature Publishing Group.
5. UniEmbed: A Unified Approach for Detecting XSS and SQL Injection Attacks Using Machine Learning Classifiers. *ResearchGate*.
6. OWASP Foundation. OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
7. Kaggle Datasets: SQL Injection Dataset and Cross-Site Scripting (XSS) Dataset. <https://www.kaggle.com/>